

ACOL 226

15th April

Built-in types

Int, Float, Char, Bool

A $\rightarrow$ B

length :: [Int] $\rightarrow$ Int

↓
type of
the length
function

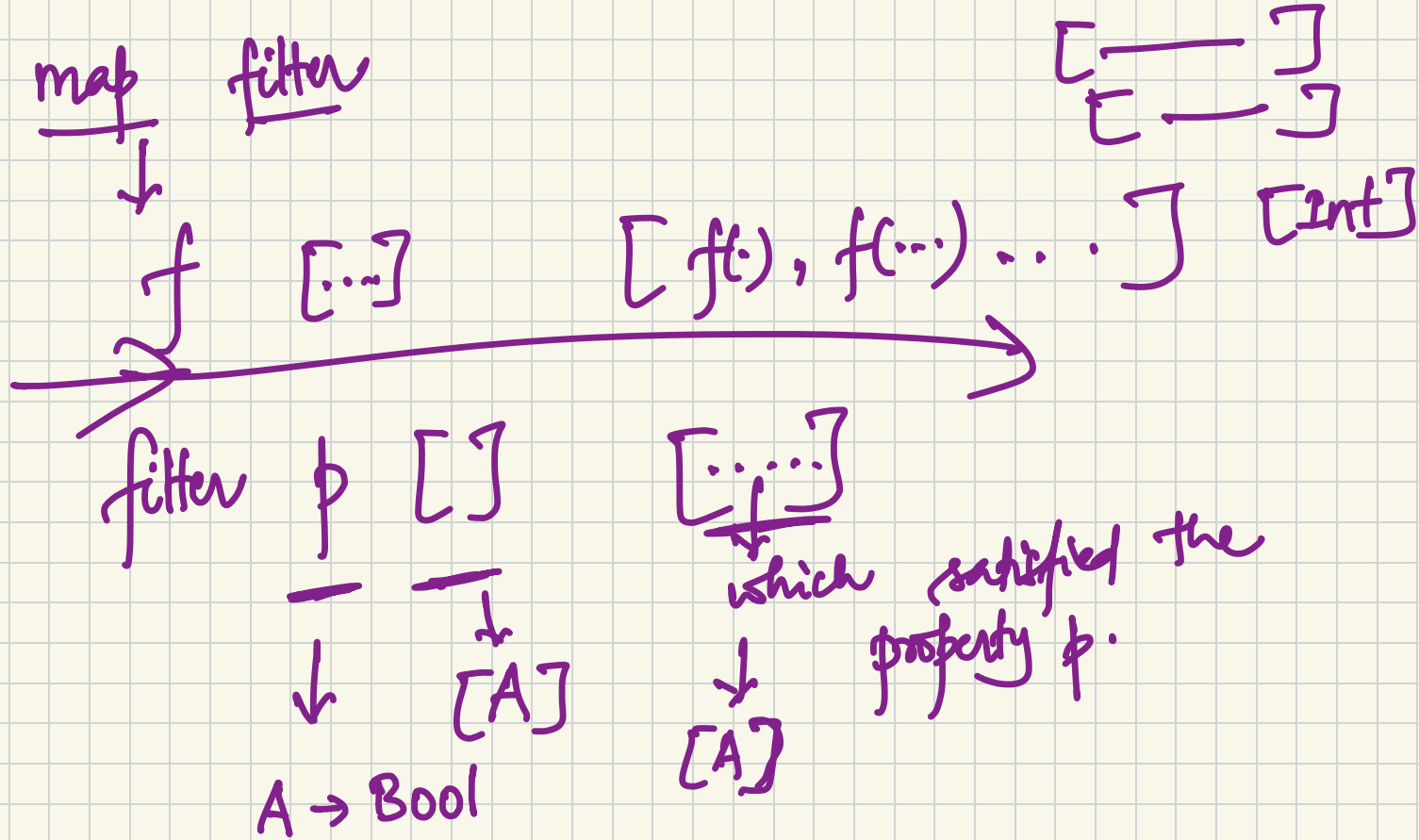
[1, 2, 3]

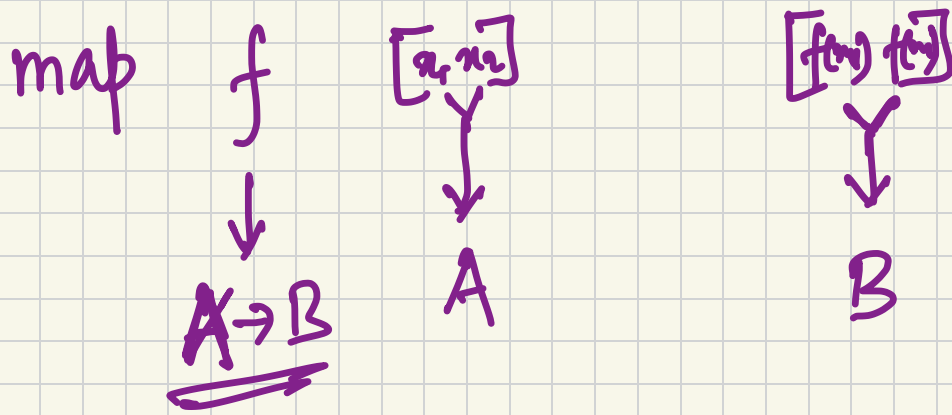
[Int]

[True, True, False]

[Bool]

[1, 2, 3] :: (2: (3: []))





$$(A \rightarrow B) \rightarrow [A] \rightarrow [B]$$

$$\text{filter} :: (A \rightarrow \text{Bool}) \rightarrow [A] \rightarrow [A]$$

$$\underline{\underline{(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]}}$$

⌋

Poly morphism

In Haskell, it is possible to say that a function works for multiple types by using type variables.

length :: [a] → Int
 ↑
 type variable

[Int]
[Float]

for any underlying type a , length is of the type $[a] \rightarrow \text{Int}$.

Functions that work the same way for different types are called polymorphic.

("takes different forms")

The diagram shows two addition operations. The first is $2 + 3$, which is circled and underlined. A curved arrow points from this expression down to the second expression, $2.3 + 3.67$, which is also underlined. To the right of these expressions, the number 5.97 is written and underlined, with a summation symbol Σ above it. A large curly bracket on the right side groups the two expressions and the result.

overloading ←

"hello" + "world"
= "helloworld"

Conditional polymorphism

$$\| \text{sum } [] = 0$$

$$\| \text{sum } (x:xs) = x + (\text{sum } xs)$$

$$\text{sum} : [\text{Int}] \rightarrow \text{Int}$$

$$\text{sum} : [\text{Float}] \rightarrow \text{Float}$$

→ $\text{sum} : [a] \rightarrow a$, provided type a supports $+$

quicksort :: [Int] → [Int]

fquicksort :: [Float] → [Float]

quicksort :: [a] → [a],

provided we
can compare values
of type
a

plus2 :: Int → Int → Int

max :: Int → Int → Int

[plus2, max]

$\text{Sumv} :: (\text{Num } a) \Rightarrow [a] \rightarrow a$

Where Num is the collection of
all types that support
numeric operations
(eg. +, *, -)

If a is in
Num, then
sumv is of the
type $[a] \rightarrow a$.

Ord is the collection
of types that
support comparison
function $< \leq$
 $> \geq$

|| quicksort :: (Ord a) $\Rightarrow$ $[a] \rightarrow [a]$
← If a is in Ord then quicksort is of the type $[a] \rightarrow [a]$.

Ord :: (a) $\rightarrow$ Bool

Ord $a \rightarrow$ denotes whether a belongs to Ord.

Eq $\rightarrow$ the set of all types
that support checking for
equality.

~~$\text{elem } x \text{ } []$~~

$\text{elem} :: (\text{Eq } a) \Rightarrow \underline{a} \rightarrow \underline{[a]} \rightarrow \text{Bool}$

$\text{elem } x [] = \text{false}$

$\text{elem } x (y:ys) \mid (x == y) = \text{True}$
 $\mid \text{otherwise} = (\text{elem } x \text{ } ys)$